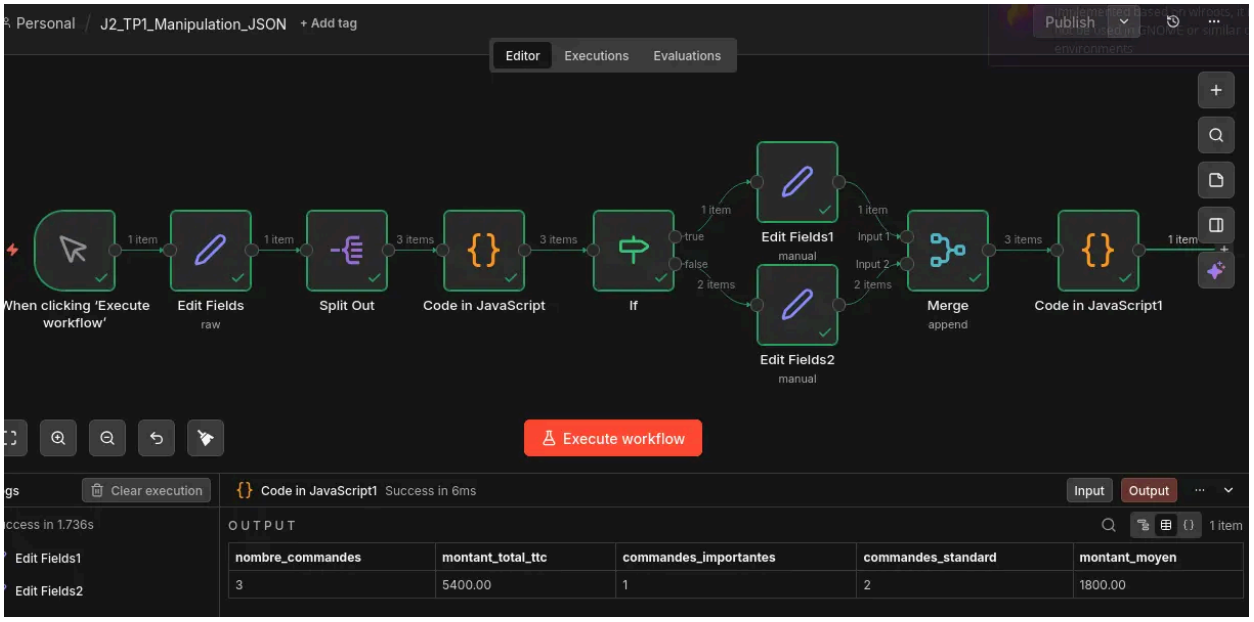
TP jour 2 - Introduction a l'I.A et Automatisation

TP 1 : Manipulation JSON et transformation de données



- Quelle est la différence entre Split Out et Loop Over Items ?
  - Split out va permettre de de créer un tableau d'items a partir de la propriété d'un objet JSON contenant un tableau alors que Loop Over Items va itérer sur une liste d'items.
- Comment gérer des structures JSON imbriquées plus complexes ?
  - Pour gérer des structures JSON imbriquées plus complexe on peut utiliser du code javascript directement.

TP 2 : Logique conditionnelle avancée avec Switch

Code js pour le résumé

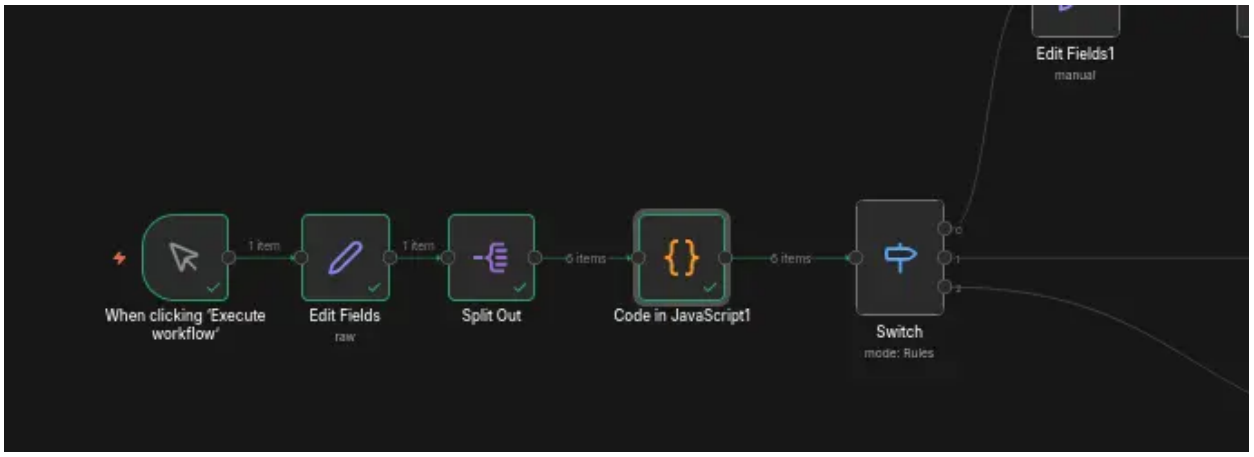
```
const priorityItemCount = []
for (const item of $input.all()) {
  const data = item.json
  let prevPriorityIndex = priorityItemCount.findIndex(p => p.name ===
data.priorite)
  if (prevPriorityIndex === -1) {
    const nextPriority = {
      name: data.priorite,
      count: 1
    }

    priorityItemCount.push(nextPriority)
    continue
  }

 const prevPriority = priorityItemCount[prevPriorityIndex]
 priorityItemCount[prevPriorityIndex] = { ... prevPriority, count:
prevPriority.count + 1 }
}

return priorityItemCount;
```

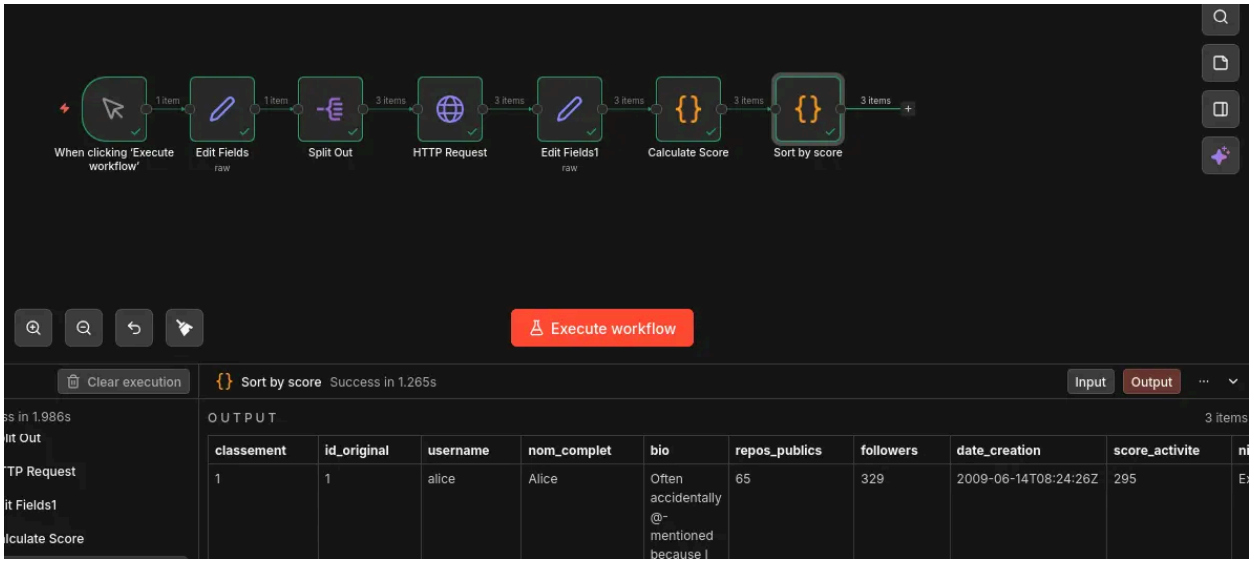
detection de mots clef dans le sujet:



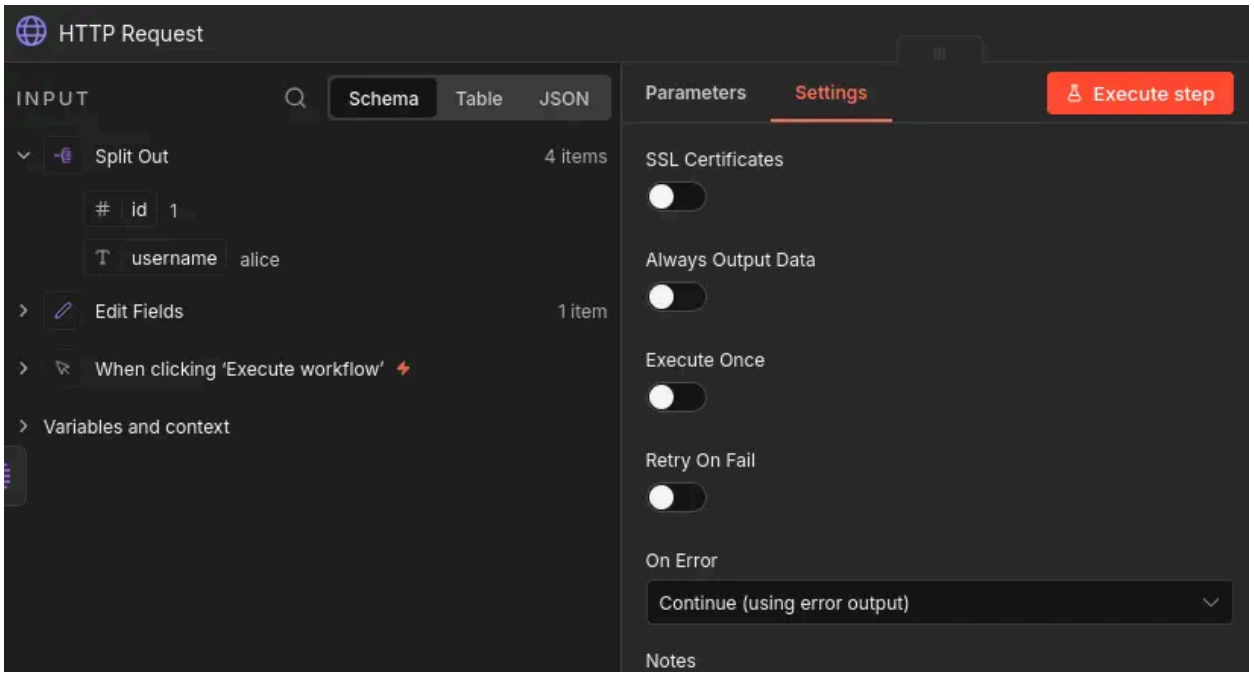

```
const priorities = ["bas", "moyen", "urgent"]

const subject = $input.item.json.sujet
const regex = /down|urgent|panne/
if (regex.test(subject)) {
  const prevPriorityIndex = priorities.findIndex(p => p ===
$input.item.json.priorite)
  const nextPriorityIndex = prevPriorityIndex + 1
  const nextPriority = nextPriorityIndex < priorities.length - 1 ?
priorities[nextPriorityIndex] : priorities[prevPriorityIndex]
  $input.item.json.priorite = nextPriority
}
return $input.item;
```

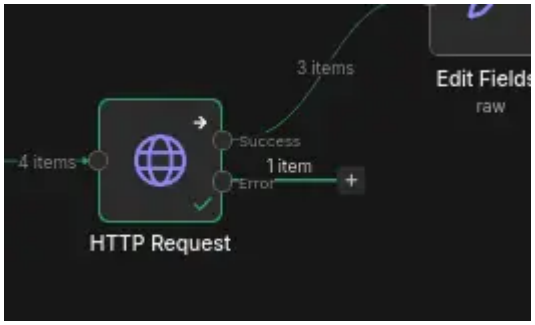
TP 3 : Boucles et itérations avec Loop Over Items



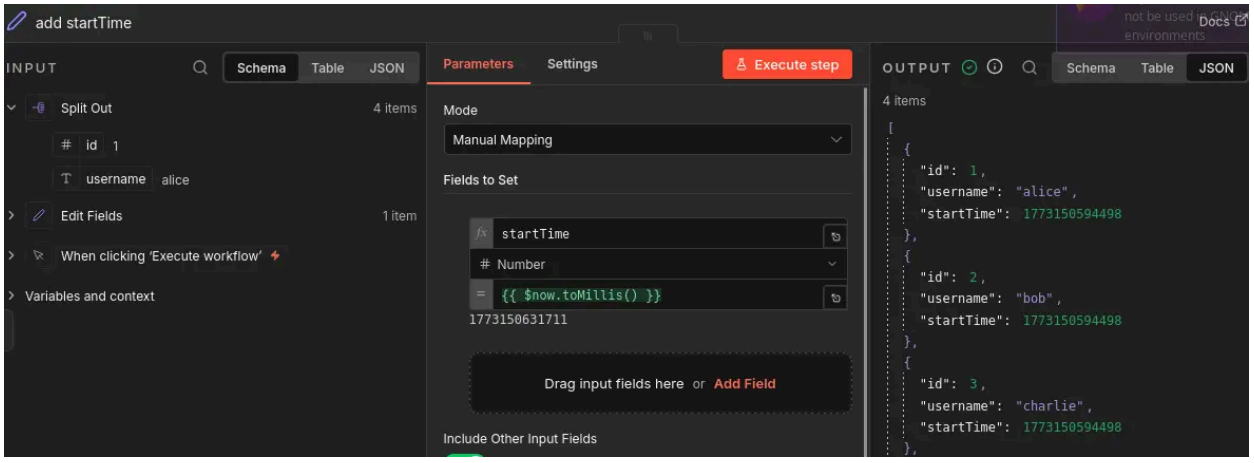
Ajoutez une gestion d’erreur si un username n’existe pas sur GitHub

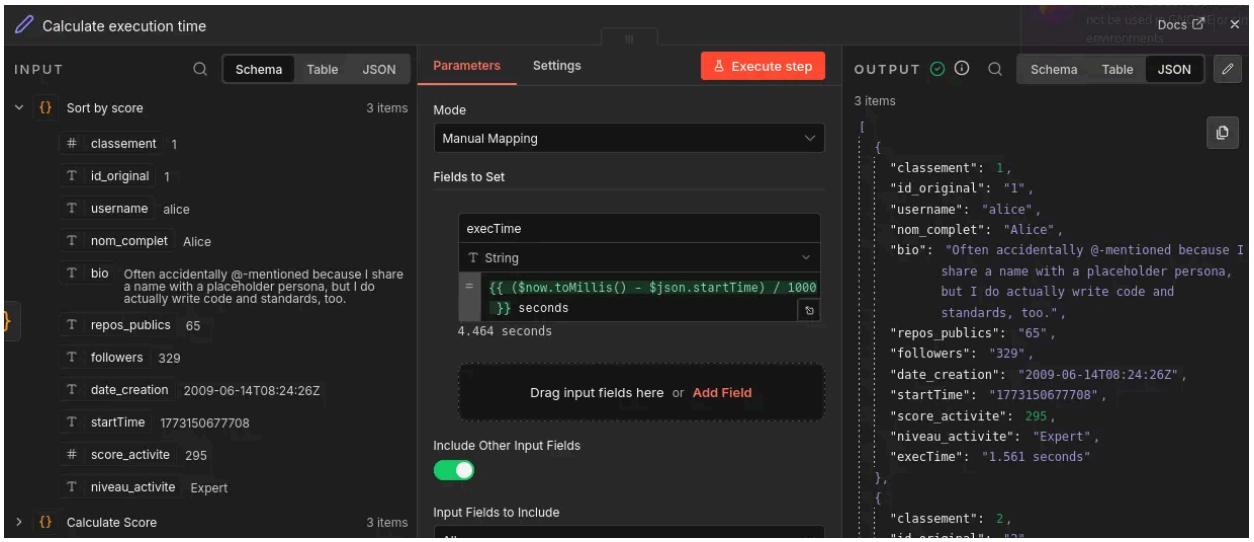


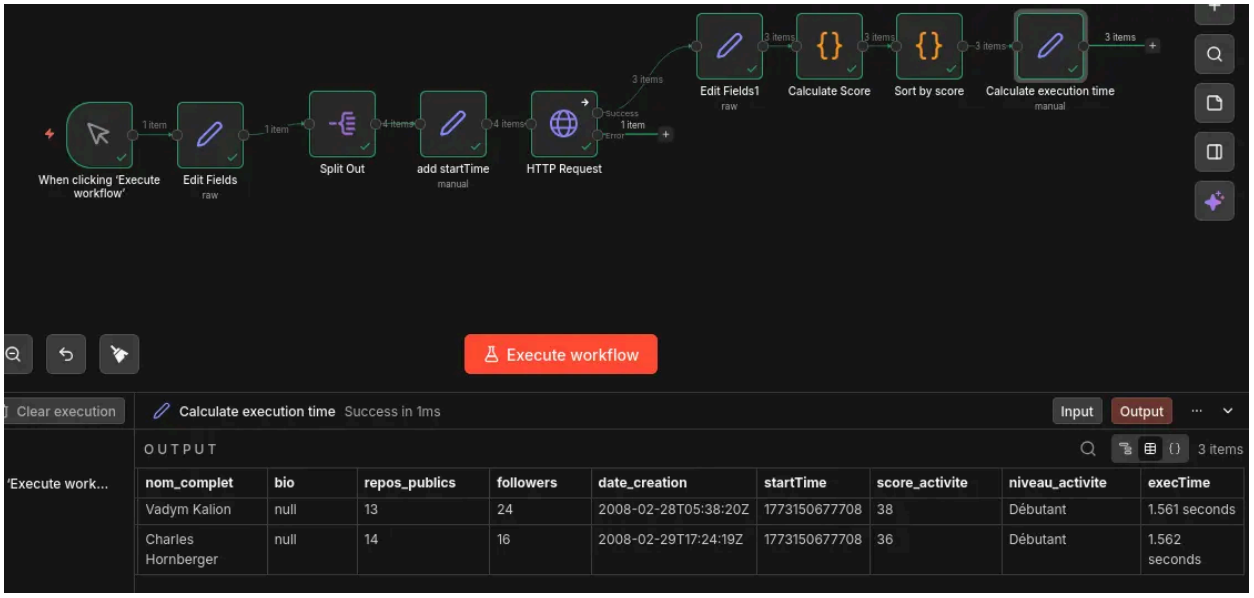
on peut utiliser l'output "error"



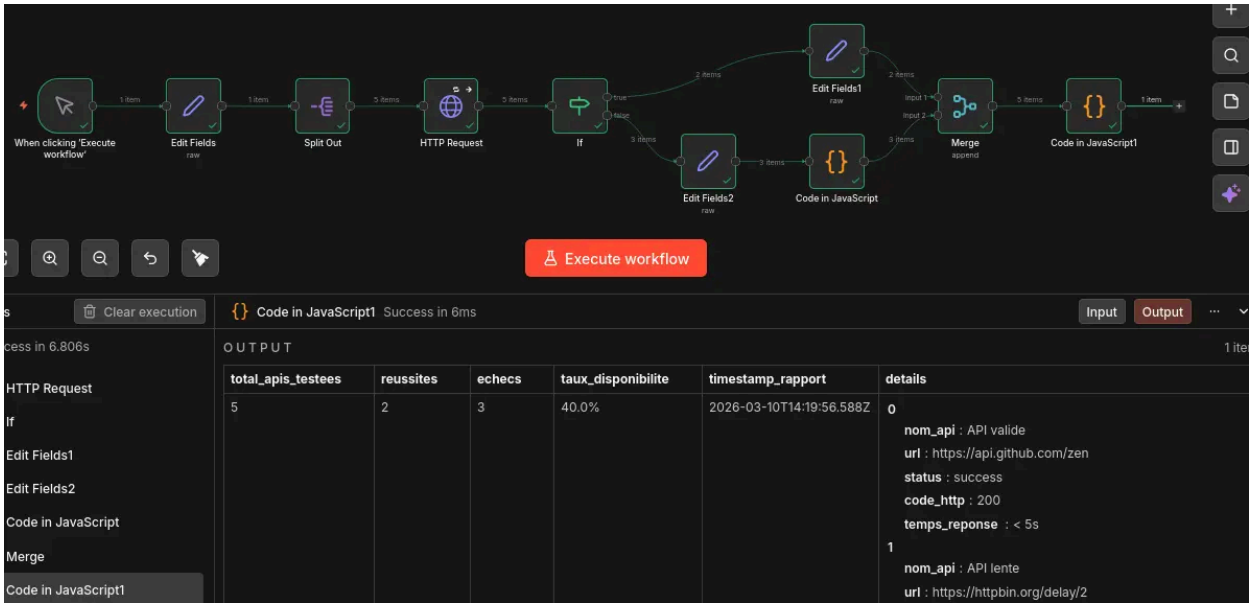
Calculez le temps moyen de traitement par utilisateur







TP 4 : Gestion d'erreurs robuste



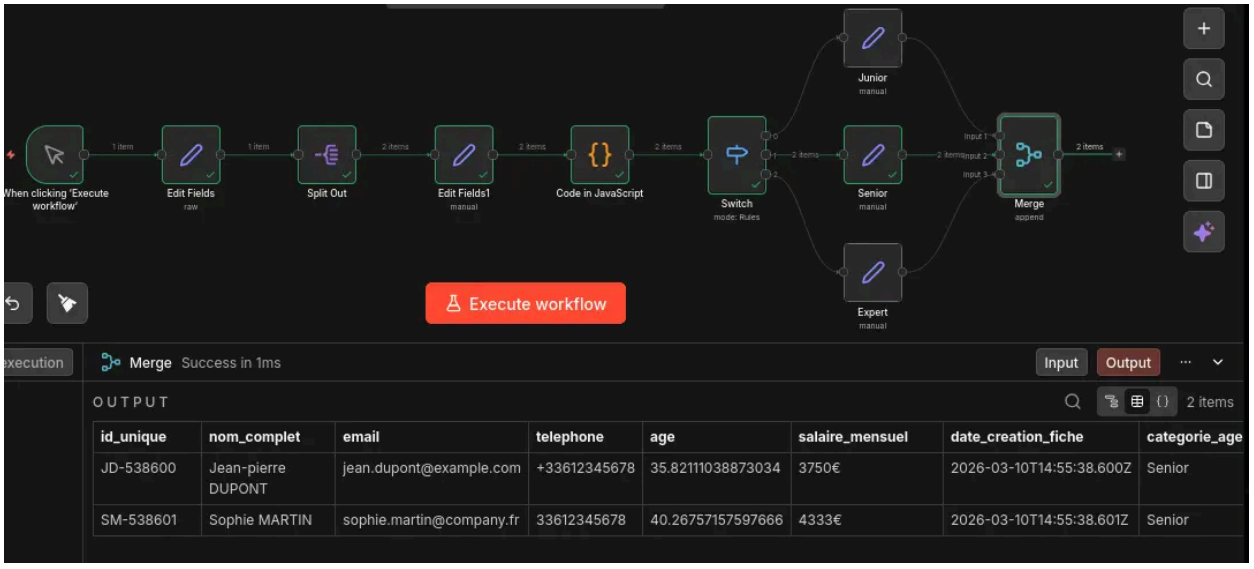
Pourquoi est-il important d'activer "Continue On Fail" ?

Pour eviter d'interompre le workflow en cas d'erreur et continuer l'execution pour les autres items.

Dans quels cas utiliser un retry vs un fallback ?

- **Retry** : Réessayer la même opération (utile pour erreurs temporaires)
- **Fallback** : Utiliser une alternative (API de secours, données en cache) quand l'opération principale échoue définitivement

TP 5 : Expressions n8n avancées



Validation Email

```
const data = $input.item.json;
const initiales = data.prenom_formate.charAt(0) +
data.nom_formate.charAt(0);
const timestamp = Date.now().toString().slice(-6);
const id_unique = `${initiales}-${timestamp}`;

// Validation email avec regex
const emailValide = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(data.email_normalise);

return {
json: {
id_unique: id_unique,
nom_complet: `${data.prenom_formate} ${data.nom_formate}`,
email: data.email_normalise,
email_valide: emailValide ? "Oui" : "Non",
telephone: data.telephone_formate,
age: data.age,
salaire_mensuel: `${data.salaire_mensuel}€`,
date_creation_fiche: new Date().toISOString()
}
};
```

TP6 : Merge et fusion de données

Calculer les statistiques par client

```
const clientStats = []

for (const item of $input.all()) {
  const current = item.json
  let prevClientIndex = clientStats.findIndex(c => c.client_id ===
current.client_id)
  if (prevClientIndex === -1) {
    clientStats.push({client_id: current.client_id, ca_annuel:
current.ca_annuel, nom: current.nom, secteur: current.secteur, nombre_commandes:
current.commande_id ? 1 : 0, total_commandes: current.montant || 0 })
    continue
  }

  clientStats[prevClientIndex] = {
     ... clientStats[prevClientIndex],
    nombre_commandes: clientStats[prevClientIndex].nombre_commandes + 1,
    total_commandes: clientStats[prevClientIndex].total_commandes +
current.montant
  }
}

return clientStats.map(c => {
  const panierMoyen = c.nombre_commandes > 0 ? c.total_commandes /
c.nombre_commandes  : 0;

  return {
    json: { ... c,
    panier_moyen: Math.round(panierMoyen),
    taux_conversion: `${((c.total_commandes / c.ca_annuel) * 100).toFixed(2)}%`}
  }
});
```